SAIL: Scalable In-Network-Assisted Loss Recovery for Cross-Datacenter RDMA

Xiao Li, Haoyuan Yu, Yuan Yang, Mingwei Xu
Tsinghua University

Abstract

Cross-datacenter transmission is critical to many emerging Internet applications, and RDMA is a promising technology. To enable efficient cross-datacenter RDMA, loss recovery is essential due to lossy long-haul links. Most existing approaches face a scalability challenge: limited resources cannot accommodate loss recovery for networks with longer links, higher bandwidth, and more concurrent flows. We present SAIL, a scalable in-network-assisted loss-recovery mechanism for cross-datacenter RDMA. SAIL decomposes selective repeat (SR) into loss detection, out-of-order packet buffering, and retransmission, which are deployed solely on DCI switches and end-host software, respecting resource constraints while coordinating to achieve provably effective SR. We propose a suite of techniques to ensure SAIL's effectiveness and scalability, including loss detection through an aggregated macro-flow and metadata piggybacking, out-of-order packet reception via dummy packet injection, and precise data repair and completion validation with minimal CPU overhead. We implement a SAIL prototype using a Tofino 1 programmable switch and a user-level software agent. Extensive experiments and simulations validate SAIL's effectiveness across a wide range of network environments. Notably, SAIL achieves 13% to 8.8× higher goodput than the state-of-the-art approaches at a 1% loss rate.

CCS Concepts: • **Networks** → **Network protocols**; **Data center networks**; **In-network processing**.

Keywords: RDMA, Cross-datacenter networks, Loss recovery

ACM Reference Format:
Xiao Li, Haoyuan Yu, Yuan Yang, Mingwei Xu. 2026. SAIL: Scalable In-Network-Assisted Loss Recovery for Cross-Datacenter RDMA. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15–18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3828523.3857022



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, China

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857022

1 Introduction

Fast-growing Internet applications such as distributed LLM training, geo-distributed storage, disaster recovery, and multi-region services are driving the demand for high-speed cross-datacenter transmission [7, 8, 32, 41, 46]. Remote Direct Memory Access (RDMA), with its high throughput, low latency, and minimal CPU overhead, is widely deployed in modern datacenter networks and is also regarded as a promising technology for cross-datacenter transmission. These benefits stem from offloading network processing to the RDMA-enabled Network Interface Controller (RNIC). Although long-haul propagation limits the benefit of reducing software processing delay, this offloading still helps sustain high throughput while preventing network processing from contending with application computations for CPU cycles, which is particularly valuable for bandwidth-intensive applications.

Extending RDMA to cross-datacenter scenarios requires accommodating the long-haul network environment, where packet loss–whether due to congestion or physical failures–is commonplace [10, 11, 18, 55], unlike in intra-datacenter networks. While efforts have been made to address loss recovery for long-haul RDMA [21, 29, 31], most existing approaches suffer from a fundamental scalability limitation: limited resources cannot scale to loss recovery for networks with longer links, higher bandwidth, and more concurrent flows. Specifically, Go-Back-N (GBN) incurs redundant retransmissions of at least the bandwidth-delay product (BDP). For selective repeat (SR), RNICs and switches lack the memory to buffer the out-of-order packets at BDP scale, whereas end hosts lack the CPU capacity to sustain per-packet processing at high data rates.

In this paper, we tackle the loss-recovery problem for long-haul RDMA and propose SAIL, a scalable SR solution enabled by network-host coordination. SAIL is deployed on datacenter interconnection (DCI) switches and end host software, requiring no changes to other components such as top-of-rack (ToR) switches and RNICs. Despite its deployment simplicity, SAIL achieves high goodput across a wide range of loss rates in different network environments. We begin by reviewing existing approaches to this problem and presenting our key insights. We then motivate the design of SAIL and summarize its contributions in overcoming several challenges.

1.1 Long-Haul RDMA Makes Packet Loss Expensive

Unlike intra-datacenter networks, long-haul links provide an environment with a much higher risk of packet loss, whether due to congestion, physical impairments, or transient failures. A straightforward method to prevent packet loss is to extend the well-known Priority Flow Control (PFC) [14] to long-haul links. PFC requires each switch to maintain a buffer large enough to absorb in-flight packets, thereby eliminating the risk of packet loss. The required buffer size approximates the BDP, which entails a prohibitively high cost for commodity switches on long-haul links. For example, a 400 Gbps link with a 10 ms round-trip time (RTT) has a BDP of approximately 500 MB, far exceeding typical on-chip buffer capacity, i.e., 20-200 MB in total [25]. Moreover, despite the performance degradation, PFC is only effective against congestion-induced packet loss and offers no protection against random losses. Therefore, an efficient loss-recovery mechanism for cross-datacenter RDMA is essential.

The most commonly deployed RDMA transport mode is Reliable Connection, which uses a simple Go-Back-N (GBN) loss-recovery mechanism to reduce hardware cost. In particular, the receiver RNIC tracks only the next expected packet sequence number (PSN). Upon detecting a gap, it discards all subsequent packets and requests retransmission starting from the missing PSN. Consequently, a single loss can invalidate up to one BDP of otherwise useful in-flight packets. On the aforementioned long-haul link, this implies retransmitting up to 500 MB of data due to a single lost packet. The redundant traffic, which scales with bandwidth and latency, increases flow completion time (FCT), wastes long-haul link bandwidth, and further exacerbates congestion.

1.2 Existing Selective Repeat Mechanisms Fall Short

To address the inefficiency of GBN's redundant retransmissions, selective repeat (SR) has attracted considerable attention. Figure 1 depicts the basic SR mechanism for cross-datacenter transmission, where two geographically distributed datacenters are connected through DCI switches across a long-haul link. Upon a packet loss on the long-haul link, some component on the receiver side must detect the loss, and out-of-order packets at BDP scale must be buffered until the missing packet is retransmitted.

In general, SR typically incurs larger buffer requirements and more complex control logic. Existing designs primarily explore three directions, each shifting the burden to a different network component. However, each direction suffers from limitations in the context of long-haul links.

RNIC-based SR: Mittal et al. [33] proposed improved RoCE NIC (IRN), which enables SR on the RNIC. Specifically, the receiver RNIC maintains a fixed-size per-flow bitmap to track out-of-order arrivals, and the sender's transmission window is constrained to the bitmap width, preventing packets from exceeding the bitmap range. Several current commodity RNICs realized similar mechanisms, such as ConnectX-6Dx [40], ConnectX-7 [35], and BlueField-3 [37]. However, scarce on-chip SRAM severely limits the achievable window size. For instance, the ConnectX-6 Dx NIC supports a maximum transmission window of merely 512 packets [40]. With a typical path MTU (PMTU) of 1024 bytes, this window covers only 512 KB of in-flight data, which falls three orders of magnitude short of 500 MB. Some approaches, such as MELO [30] and FaSR [13], allocate bitmap entries on demand from a shared bitmap pool. This improves average SRAM utilization, but the required resources still grow with the recovery window and the number of concurrent losses, leaving the fundamental long-haul scalability problem unresolved.

Software-assisted SR: Some approaches leverage software on end hosts to enable SR, exploiting the abundant host memory and flexible state management. Typically, Flor [24] and UCCL [57] use the Unreliable Connection transport mode of the RNIC, and implement reliability purely in software. While this removes the RNIC bitmap limitation, host software must process packets at fine granularity to track delivery and drive retransmissions. As a result, the host CPU remains involved in the high-speed packet processing even in the absence of packet loss, increasing overhead and limiting throughput. To address the issue, some approaches propose hybrid designs that retain fast packet processing in hardware while offloading SR state to software, such as SRNIC [52] and OmniDMA [31]. However, they require changes to the RNIC architecture and therefore cannot be deployed on existing commodity RNICs.

In-network SR: Some approaches leverage switches to observe and process losses before packets reach the end-host RNIC. NetRT [51] proxies SR between top-of-rack (ToR) switches: the sender ToR caches packet replicas, while the receiver ToR buffers out-of-order packets. However, both switches require BDP-scale buffering, which can easily exceed the total on-chip buffer capacity, even before accounting for sharing across ports and queues. LR2 [29] filters redundant retransmissions at the sender-side DCI switch, but the receiver-side DCI switch still maintains a BDP-scale reordering pool. Moreover, the redundant packets retransmitted by the RNIC increase the risk of intra-datacenter congestion. Although switches with sufficient off-chip memory could make such BDP-scale buffering feasible, releasing the buffered out-of-order packets can still take considerable time, substantially increasing FCT and potentially causing transient congestion.

1.3 Key Insights and Motivation

Based on a thorough analysis of existing approaches, we find that enabling SR involves three basic functions: detecting losses, preserving out-of-order packets, and performing retransmission, as shown in Figure 1. Consolidating all three functions entirely within a single component is inadvisable,

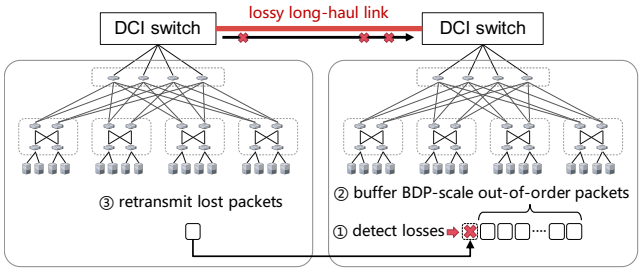


Figure 1. Cross-datacenter RDMA network topology and the basic selective repeat mechanism.

as it would be constrained by that component's limited resources and processing capacity. Specifically, RNICs have limited SRAM, and modifying RNIC hardware incurs high redeployment costs; switches cannot buffer packets for long periods due to their limited on-chip capacity; and processing every packet in host software incurs high CPU overhead.

A natural solution is to judiciously distribute the functions and coordinate across different components. In particular, the end host is best suited for BDP-scale buffering owing to its abundant memory, and other functions should be offloaded to avoid high-speed packet processing at end hosts. However, per-flow state management remains challenging as the number of concurrent flows scales. Existing approaches resort to modifying RNICs [31, 52] which need only handle a small number of flows, yet this creates the burden of updating a large number of RNICs in an operational datacenter.

In this paper, we pursue a different direction: offloading part of the SR functions to DCI switches, as they directly observe the long-haul link before the RNIC reacts and only a small number of switches require modification. The DCI switch and host software then cooperate to handle retransmissions upon packet loss. We leave the RNIC hardware unmodified and on the common-case fast path, while ensuring that the end host software is invoked only for a small number of retransmissions to recover lost data, thereby removing the overhead of common-case data processing from the software path.

1.4 Challenges and Contributions

Building on the insights and motivation above, we develop SAIL, a scalable in-network-assisted loss-recovery mechanism for cross-datacenter RDMA. We identify the challenges and present SAIL's technical contributions to address them.

First, a practical recovery mechanism should detect packet losses with switch state that is independent of the number of concurrent flows. Existing approaches maintain per-flow context for loss detection throughout the lifetime of each RDMA connection [13, 29, 33, 51]. If deployed at the datacenter boundary, this would require DCI switches to track all cross-datacenter flows, rapidly depleting data-plane SRAM. SAIL addresses this issue by aggregating all flows into a macro-flow at the sender-side DCI (S-DCI), and identifying the lost packet via metadata piggybacked on data packets at the receiver-side DCI (R-DCI). Our method achieves consecutive-loss detection with $O(1)$ space complexity, $O(1)$ time complexity, and minimal extra bandwidth consumption.

Second, our mechanism must preserve valid packets arriving after a loss rather than discarding them, yet this must be achieved without modifying RNIC hardware that either discards post-gap packets (GBN) or supports only a limited out-of-order reception window (newer SR-capable RNICs). SAIL addresses this issue by injecting a dummy packet with a header identical to that of the lost packet, and ensuring it arrives at the RNIC in the correct position. Consequently, the RNIC perceives no packet loss and stays on the common-case fast path, while end-host software repairs the dummy data.

Third, host software must precisely repair the corrupted memory regions (i.e., the dummy data) and determine when completed operations can be safely exposed to applications, all with minimal CPU overhead to preserve RDMA's offloading benefits and despite the RNIC's inability to guarantee correct delivery. To address this challenge, the R-DCI notifies the receiver-side software agent of a packet loss to trigger a NACK, which is then used to locate the source address range at the sender. The R-DCI also generates a message-end notification to signal that all losses for a message have been reported, enabling the receiver to validate message completion.

The contributions of the paper are as follows.

- Following the design principle of decomposing SR, we present SAIL, a scalable in-network-assisted loss recovery mechanism for cross-datacenter RDMA. SAIL achieves provably effective SR requiring deployments only on DCI switches and end-host software.
- We propose a suite of techniques to ensure the effectiveness and scalability of SAIL, including loss detection through aggregated macro-flow and metadata piggybacking, out-of-order packet reception via dummy packet injection, and precise data repair and completion validation with minimal CPU overhead.
- We implemented a SAIL prototype using a Tofino 1 programmable switch [15] and a user-level software agent. The source code is publicly available at GitHub [42].
- We evaluate SAIL through extensive testbed experiments and large-scale simulations, against several state-of-the-art approaches. The results demonstrate SAIL's effectiveness across diverse loss rates and network environments. Notably, SAIL achieves 8.8× higher goodput than GBN and 13% higher goodput than Flor [24] at a 1% loss rate.

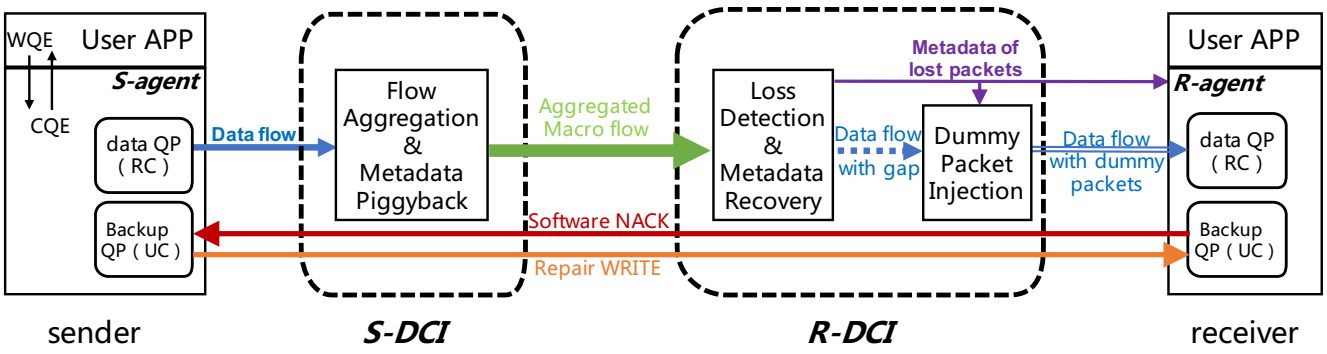


Figure 2. SAIL's host-network collaborative selective-repeat workflow.

2 SAIL Design

2.1 Overview

SAIL preserves the common-case RDMA data path while decomposing loss recovery across four components: the sender-side DCI switch (*S-DCI*), the receiver-side DCI switch (*R-DCI*), the sender host software agent (*S-agent*), and the receiver host software agent (*R-agent*). Figure 2 illustrates the architecture of SAIL, with commodity RNICs left unmodified and running in default GBN recovery mode.

On the end hosts, applications communicate over unmodified Reliable Connection (RC) data QPs. The *S-agent* and *R-agent* establish a backup Unreliable Connection (UC) QP for each RC data QP. Upon detecting a packet loss, the *R-agent* sends a NACK to the *S-agent* through the backup QP; the *S-agent* then initiates data repair by posting a `WRITE_WITH_IMM` work request, likewise via the backup QP. The *R-agent* also handles completion validation, after which the data is exposed to applications.

The *S-DCI* and *R-DCI* switches are implemented on programmable switches. The *S-DCI* switch aggregates data flows into a macro flow, maintains $O(1)$ metadata for loss detection, and piggybacks metadata on each data packet. The *R-DCI* switch de-aggregates the macro flow and identifies lost packets using piggybacked metadata and an $O(1)$ state. Upon loss detection, the *R-DCI* switch simultaneously notifies the *R-agent* and injects a dummy packet to fill the gap in the data flow.

From the processing-flow perspective, SAIL handles packet losses on the long-haul link through a three-stage process. First, the *S-DCI* and *R-DCI* switches enable a scalable loss identification on an aggregated macro flow, eliminating the need for per-flow state (§2.2). Second, the receiver host buffers out-of-order packets without disrupting the RNIC's common-case data path, using dummy packet injection at the *R-DCI* switch (§2.3). Finally, the dummy-filled data is repaired via the backup UC QP, and completion is exposed to the application only upon validation that the message has been fully repaired (§2.4). The remainder of this section elaborates on these three stages.

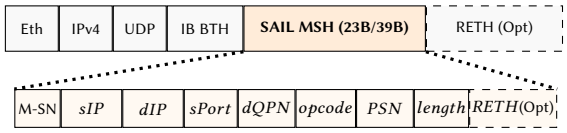


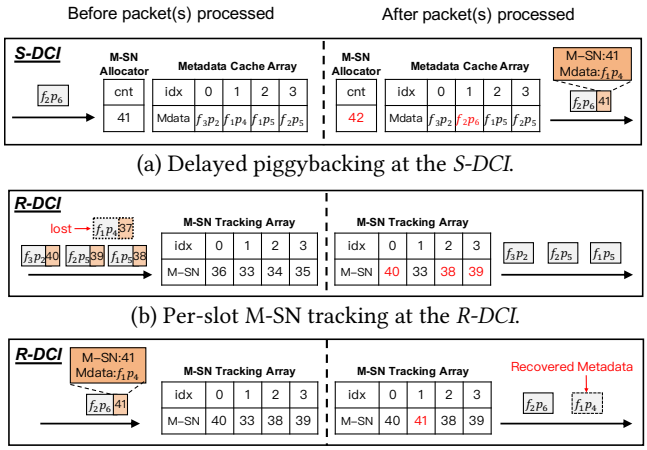
Figure 3. SAIL Macro-Sequence Header (MSH) format. MSH carries the current packet's M-SN and piggybacked recovery metadata for a previous packet. The destination UDP port is not included because it is fixed to 4791 for RDMA.

2.2 In-Network Loss Identification

We consider multiple flows that are transmitted from one data center to another through a single path. We need to detect lost packets on the *R-DCI* switch. As discussed above, SAIL aggregates these cross-datacenter flows into a single macro flow to ensure scalability. This can be realized by simply inserting a macro-sequence number (M-SN) into the packets. We introduce a customized header called Macro-Sequence Header (MSH). An MSH is inserted after the InfiniBand Base Transport Header (BTH) by the *S-DCI* switch, and is removed by the *R-DCI* switch to restore the original packet.

However, simply checking the continuity of M-SN is far from sufficient. The problem is to identify the lost packet, which means determining the packet fields that are necessary for dummy packet fabrication (§2.3) and precise retransmission (§2.4). These fields include the five-tuple, destination QPN, opcode, PSN, and payload length. Note that nearby packets may belong to different flows and do not help. One possible solution is to maintain states for each flow on the *R-DCI* switch, but this does not scale and defeats the purpose of using macro-flow. To address the issue, we propose a method with $O(1)$ space complexity and $O(1)$ time complexity.

The key of our method is to piggyback the necessary fields as metadata by another packet. In the simplest case, the *S-DCI* switch just stores the metadata of the last packet. Each incoming packet piggybacks the metadata of the last packet, and leaves the metadata of itself on the *S-DCI* switch. When



(a) Delayed piggybacking at the *S-DCI*.

(b) Per-slot M-SN tracking at the *R-DCI*.

(c) Loss identification at the *R-DCI*.

Figure 4. Example of SAIL's loss identification with delayed metadata piggybacking ($M = 4$). The vertical dashed line separates the switch state before and after processing the shown packet(s). Note that, at the *R-DCI*, processed packets may still be buffered for the ordered release mechanism in §2.3 rather than immediately leaving the switch.

a single packet loss occurs, the *R-DCI* can detect it immediately by checking the M-SN, and identify the lost packet by the metadata carried by the next packet, i.e., the packet with an M-SN that is 2 greater than the last M-SN received. This method introduces an extra bandwidth consumption to transmit the metadata. However, the size of the metadata is just 23 bytes, which is small compared to the typical RDMA packet size (1082 bytes [38]). In a special case of an initial packet of an RDMA WRITE message (*e.g.*, `RC_WRITE_FIRST` or `RC_WRITE_ONLY`), parameters of Virtual Address (VA) and R_Key must be piggybacked, but the overhead is also small. The complete MSH structure is shown in Figure 3.

To deal with consecutive packet loss, we extend the above method and propose "delayed-piggyback". In particular, the *S-DCI* stores the metadata of the last M packets, where M is a small constant integer greater than 1. For each incoming packet whose M-SN is seq, the *S-DCI* switch finds the metadata stored in the (seq mod M)-th slot, and piggybacks the metadata in the packet. The metadata of this packet is then stored in the same slot. At the receiving end of the cross-datacenter path, the *R-DCI* checks the M-SN of each packet and records the missing ones. Let seq_m be the M-SN of a missing packet. The metadata of this packet is carried by the packet with the M-SN $seq_m + M$. As a result, a burst loss of M packets at most can be detected correctly, with the precise identities of all missing packets captured. In the case where both packets with M-SN seq_m and M-SN $seq_m + M$ are lost, e.g., more than M consecutive packets are lost, the *R-DCI* will resort to RNIC's hardware recovery mechanism by not performing dummy packet fabrication.

Figure 4 shows an example of SAIL's loss identification with $M = 4$. In Figure 4(a), when packet f_2p_6 passes through the *S-DCI*, it is assigned M-SN 41. Since 41 mod 4 = 1, the *S-DCI* reads the metadata stored in slot 1 and piggybacks it in the packet header. It then updates slot 1 with the metadata of the current packet. Under delayed piggybacking, the metadata carried by M-SN 41 corresponds to M-SN $41 - M = 37$. Suppose that the packet with M-SN 37 is lost on the long-haul link. As shown in Figure 4(b), before M-SN 41 arrives, packets with M-SNs 38, 39, and 40 have already arrived and updated their tracking slots at the *R-DCI*, while slot 1 still records M-SN 33 because M-SN 37 is missing. In Figure 4(c), when M-SN 41 arrives, the *R-DCI* observes that slot 1 jumps from M-SN 33 to M-SN 41. It then infers that M-SN 37 is missing and uses the metadata piggybacked by M-SN 41 to identify the lost packet as f_1p_4.

Essentially, our in-network loss detection trades a modest increase in bandwidth for $O(1)$ storage overhead. The delayed piggyback method actually introduces little delay in the detection time because M is small and the data rate is very high. In an environment that is highly lossy, we can use a greater M or further extend SAIL to enable multi-slot piggyback, which means the metadata of the current packet will be carried by multiple packets with M-SNs $seq + k \times M$ ($k = 1, 2, \ldots$). This increases bandwidth consumption and detection latency a little, yet exponentially reduces the probability of metadata loss. By default, the metadata of each packet is carried by one single packet.

2.3 Out-of-Order Packet Reception

Once a missing packet has been identified, SAIL must preserve the valid payloads that arrive after the loss. Some recent SR designs [25, 33, 52] avoid dedicated out-of-order buffering by placing valid post-loss payloads directly into receiver memory. They modify the RNIC logic and extend packet headers with per-packet remote memory address. SAIL achieves the same goal without RNIC modifications. The key is to make the packet stream observed by the RNIC indistinguishable from an ordinary in-order RDMA stream.

To realize the idea, we first ensure that each packet delivered to the RNIC carries a protocol-compliant header and correct flow-specific metadata, including the five-tuple, destination QPN, opcode, PSN, and payload length. To this end, once the metadata of a lost packet is available, the *R-DCI* constructs a protocol-compliant dummy packet with the same metadata and length as the missing packet. If the lost packet is the first packet of an RDMA WRITE operation, the dummy packet also carries the recovered RETH fields to direct memory placement of the post-loss payload. The dummy packet does not need to be correct in payload: it only occupies the missing PSN position and fills the corresponding destination memory region. The software agents later overwrite this region with the correct payload (§2.4).

After that, we need to ensure that the packets delivered to the RNIC are continuous in PSN order. This requires each dummy packet to be inserted in the exact loss position. Note that the *R-DCI* simply releases packets in increasing M-SN order, which preserves PSN order within each RDMA flow. Thus, we must temporarily buffer subsequent packets after a loss, because the metadata of the lost packet may arrive later due to the "delayed-piggyback". The *R-DCI* therefore maintains two queues, Q_{normal} and Q_{dummy}. Normal packets are enqueued into Q_{normal}, while dummy packets are enqueued into Q_{dummy}. Note that, under the SAIL's loss detection mechanism, dummy packets are also generated and enqueued in increasing M-SN order. The *R-DCI* also maintains a register Seq_{next}, which records the next M-SN allowed to leave. It polls the head packets of Q_{normal} and Q_{dummy} and releases the packet whose M-SN equals Seq_{next} and then advances Seq_{next}. By repeating this process, the *R-DCI* ensures that packets leave in M-SN order.

However, as discussed in §2.2, SAIL may fail to recover the metadata of a lost packet in rare cases. This can cause Seq_{next} to be stuck at the loss position and prevent later packets from leaving the switch. SAIL therefore needs to determine when the metadata of the lost packet at Seq_{next} is unrecoverable and to release queued packets in time. In such a case, the head M-SN of Q_{normal} must be larger than Seq_{next}, indicating that the packet at Seq_{next} has been lost. A simple approach is to exploit the continuity of loss detection: if Q_{dummy} is not empty and the head M-SN of Q_{dummy} is also larger than Seq_{next}, then the metadata of the lost packet is unrecoverable. However, this approach is inefficient because it must wait until a later loss is detected, inducing unpredictable delay and increasing the pressure on the switch buffer.

To avoid unnecessary waiting, the *R-DCI* maintains an additional register Seq_{bound}, which is set to the current maximum observed M-SN minus M. Since SAIL's loss detection window is M, any unresolved loss before Seq_{bound} can no longer recover its metadata. When $Seq_{bound} > Seq_{next}$, the *R-DCI* advances Seq_{next} to the minimum among Seq_{bound} and the head M-SNs of Q_{normal} and Q_{dummy}, and then resumes polling and dequeuing. As a result, the *R-DCI* skips loss positions whose metadata is unrecoverable and releases subsequent packets in time. These unrecoverable losses become visible to the RNIC, causing the system to fall back to the RNIC's native recovery mechanism, e.g., GBN. As a fail-safe mechanism, the *R-DCI* also uses a timeout to release the queues when no subsequent packets arrive to advance loss detection. The timeout path also advances Seq_{next} to prevent subsequent packets from being stuck.

2.4 Data Repair and Completion Validation

SAIL relies on software agents on the two end hosts, called the *S-agent* and the *R-agent*, to ensure that data are completely and correctly committed to applications. First, the agents are responsible for repairing memory regions occupied by dummy packets by retransmissions. Second, the agents decide when an RNIC completion queue entry (CQE) can be safely exposed to the application. Because SAIL hides losses from the RNIC, the CQE generated by the RNIC no longer proves that the original RDMA message has been delivered correctly.

Each SAIL agent runs in the same process as the original RDMA connection, with direct access to the connection context, posted Work Queue Elements (WQEs), and memory regions. For each cross-datacenter RC data connection, the agents maintain a backup RDMA connection for recovery and control messages. Following Flor [24], this backup connection uses Unreliable Connection (UC) mode. A lost UC operation does not force subsequent operations on the same QP to be discarded, allowing the agents to implement flexible software-level reliability.

Precise data repair: After receiving a loss notification from the *R-DCI*, the *R-agent* records the loss into a loss queue and sends a software NACK carrying the missing PSN to the *S-agent*. The *S-agent* derives the source address range corresponding to the missing PSN from the connection's starting PSN and the payload length specified in the WQEs, leveraging the fact that the PMTU is fixed. The *S-agent* then posts a `WRITE_WITH_IMM` work request on the backup QP to overwrite the dummy-filled destination region directly, avoiding extra data copying and kernel involvement on both ends. The immediate value of the work request carries the repaired PSN, allowing the *R-agent* to update its loss state when the repair write completes. For two-sided SEND/RECV operations, the *S-agent* cannot infer the destination address alone, so the *R-agent* also includes the matched receive-buffer context in the software NACK. Since each missing PSN is tracked independently in the loss queue, the *R-agent* can initiate a new repair without waiting for earlier repairs to complete. Multiple repairs can therefore proceed concurrently and overlap under frequent losses.

Completion validation: SAIL agents expose the RNIC CQE to the application only after validating that the entire message has been delivered without unrepaired loss. A direct approach is to let the *R-agent* check whether the loss queue contains an unrepaired PSN in the corresponding message range after the RNIC CQE is generated. This is insufficient for two reasons. First, one-sided WRITE operations do not generate receiver-side CQEs. Second, even for SEND/RECV, a loss notification from the *R-DCI* may arrive later than the RNIC CQE because the two notifications follow different paths, leading to false validation.

SAIL therefore uses a message-end notification to indicate that all losses detected for a message have been reported. The *R-DCI* generates the message-end notification when the last packet of a message traverses the switch. The message-end notification shares the same ordered path as loss notifications, ensuring that all detected losses for a message are

reported before the message-end notification arrives. Upon receiving the message-end notification, the *R-agent* checks its loss queue. If no losses remain unrepaired for the message, completion is validated immediately. Otherwise, validation is deferred until all repairs are finished. After validating completion, the *R-agent* exposes the CQE to the application for two-sided operations, and notifies the *S-agent* to trigger sender-side CQE delivery.

Correctness during GBN fallback: When *R-DCI* fails to recover the metadata of a lost packet in rare cases, it does not insert the dummy packet. The receiving RNIC therefore observes a PSN gap and invokes its native GBN recovery. Before GBN retransmission completes, the RNIC does not generate a CQE, so completion is not prematurely exposed to the application. Once the retransmitted packet arrives, SAIL resumes assisting with selective repeat from that point onward. Meanwhile, some in-flight packets discarded after GBN is triggered may contain additional gaps that have already been identified by the *R-agent* and are therefore also repaired by SAIL. This can cause the same destination memory region to be overwritten more than once, but the final data remain correct.

The software agents incur minimal overhead. The CPU overhead scales with the frequency of loss and message-end events, while the vast majority of successfully delivered packets are processed entirely by RNIC hardware. The additional software delay is also acceptable in long-haul RDMA networks, where physical propagation delay dominates.

3 Implementation

We implement a SAIL prototype that includes the Tofino 1 switch data plane and the host-side SAIL agent. The source code is publicly available at GitHub [42].

3.1 Switch Implementation

We implement SAIL's *S-DCI* and *R-DCI* prototypes on a Tofino 1 programmable switch. The switch implementation consists of ~1600 lines of P4 data-plane code and ~130 lines of C control-plane code. The *S-DCI* logic is placed primarily in the egress pipeline, where it assigns M-SNs and piggybacks delayed metadata immediately before packets enter the long-haul link. The *R-DCI* logic is placed primarily in the ingress pipeline, where it detects losses, constructs dummy packets, generates notifications, and releases packets in M-SN order. The control plane initializes data-plane tables and registers, and configures switch resources such as multicast groups, mirroring sessions, and recirculation ports.

Queue emulation. SAIL requires logical queues to temporarily hold normal packets and dummy packets before releasing them in M-SN order. However, Tofino 1 does not provide flexible queue management primitives that can be directly accessed from the P4 pipeline. We therefore emulate these logical queues using packet recirculation [17]. When packets enter a logical queue, the switch assigns them increasing queue tags that indicate their positions in the queue and then forwards them to the recirculation port. The switch also maintains a register that records the current head tag of each queue. Each time a recirculated packet re-enters the pipeline, it compares its queue tag with the head tag to determine whether it is at the queue head. Only the queue-head packet is allowed to execute the release check described in §2.3, which prevents later packets from bypassing earlier packets in the same logical queue. Newer switch ASICs with richer queue management support, such as Tofino 2 [16], can avoid the need for this recirculation-based queue emulation.

Timeout mechanism. The switch assigns a fixed recirculation budget to each packet entering a logical queue. The budget is decremented whenever the packet is recirculated. Once the budget reaches zero, the switch treats the packet as timed out and releases it. This mechanism prevents packets from being recirculated indefinitely in rare corner cases, and naturally ensures that timed-out packets are dequeued in their arrival order due to the same recirculation budget.

Dummy packet construction. When the switch obtains the metadata of a lost packet, it uses multicast to clone the current packet to the recirculation port. After the cloned packet re-enters the pipeline, the switch rewrites its headers and metadata to construct the dummy packet. This method requires the cloned packet payload to be at least as large as the dummy packet payload to be generated. This condition usually holds because most RDMA data packet payloads are PMTU-sized. Our prototype also injects a PMTU-sized template packet that continuously recirculates inside the switch. When no sufficiently large packet is available for dummy packet construction, the switch multicasts the template packet and rewrites it into the required dummy packet.

Loss and message-end notification construction. The switch uses the mirror feature [17] to generate control notifications for host agents. It mirrors and truncates the current packet to create a compact notification packet. The data plane then rewrites the notification header to carry the corresponding loss or message-completion metadata.

3.2 SAIL Agent Implementation

We implement the SAIL agent as a userspace C prototype built on the Verbs interface [36]. It consists of ~1600 lines of C code. The agent maintains backup UC connections, processes loss and message-completion notifications from the switch, triggers precise repairs, and exposes CQEs to the application.

Our current prototype implements the SAIL logic as a user-level agent for ease of development and evaluation. In a production deployment, this logic can be integrated into the RDMA user-space driver or a runtime shim, allowing SAIL to support existing RDMA applications with little or no application modification.

4 Evaluation

We evaluate SAIL using hardware testbed experiments and large-scale NS-3 [9] simulations to answer the following questions:

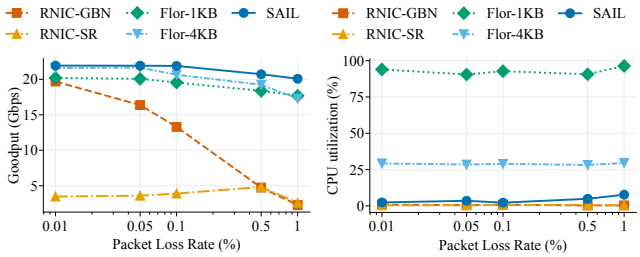
- Can SAIL sustain high long-haul goodput while keeping low host CPU overhead?
- Can SAIL reduce average and tail FCT across different scenarios?
- Can SAIL scale to larger datacenter settings with limited switch resources and little impact on intra-datacenter traffic?

4.1 Testbed Setup

Hardware topology. Our testbed consists of two Barefoot Tofino 1 programmable switches, acting as the sender-side and receiver-side DCI switches. Two servers, each equipped with one dual-port NVIDIA ConnectX-6 Dx RNIC, connect to the two switches and act as two senders and two receivers. One server is additionally equipped with another dual-port NIC and runs a DPDK-based [6] long-haul link emulator between the two DCI switches. The emulator receives packets from one port, buffers them, and forwards them to the other port after the configured delay. It also drops packets before forwarding according to the configured loss probability. Due to the PCIe bandwidth (PCIe3.0x8) of the servers and the processing capacity of the emulator, each ConnectX-6 Dx port is configured at 25 Gbps to ensure stable measurements.

Workloads and metrics. Unless otherwise specified, experiments use a 5 ms one-way long-haul delay, a 0.1% packet loss rate, and a 60% offered load. The default workload uses Poisson flow arrivals and randomly selected sender-receiver pairs. Flow sizes follow a Websearch [2] distribution: 60% of flows are no larger than 200 KB, 90% are no larger than 5 MB, and the largest flow size is 30 MB. We report goodput, host CPU utilization, flow completion time (FCT), and FCT slowdown.

Baselines. We compare SAIL with three baseline schemes. RNIC-GBN uses the default Go-Back-N recovery in commodity RNICs. RNIC-SR enables the SR mode supported by ConnectX-6 Dx and configures the send window to the maximum supported value of 512 packets. Flor represents software-based recovery [24]. It splits each message into chunks, transmits them over UC, and performs chunk-level loss detection and retransmission in host software. We evaluate Flor with 1 KB and 4 KB chunks, denoted Flor-1KB and Flor-4KB. Smaller chunks provide finer-grained repair but require more software operations, whereas larger chunks reduce software work at the cost of coarser repair. In our testbed, we also observe that chunks larger than the default 1024-byte PMTU can occasionally trigger transient receive-side stalls after losses. The exact cause is hidden inside the RNIC implementation. Therefore, the lower performance of Flor-4KB in later experiments should be interpreted as partly reflecting this hardware-specific behavior in our testbed. We do not compare with LR2 on the hardware testbed because its hardware implementation is not publicly available.



(a) Goodput (b) CPU utilization

Figure 5. Goodput and CPU utilization in the saturated single-pair setting.

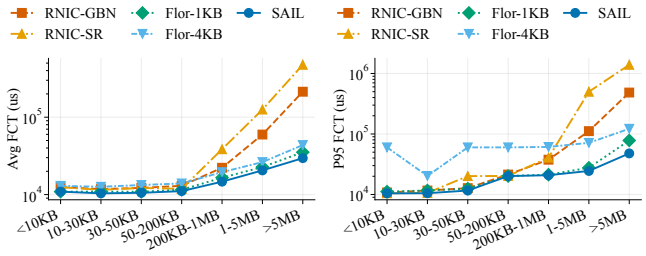
Table 1. Goodput (Gbps) under burst losses at an overall packet loss rate of 0.1%.

Burst length	Single-slot		Two-slot	
B	$M = 8$	$M = 16$	$M = 8$	$M = 16$
8	23.09	22.92	22.50	22.29
16	17.89	22.90	22.54	22.51
24	18.19	19.56	19.02	22.34

4.2 Single-Pair Throughput and CPU Overhead

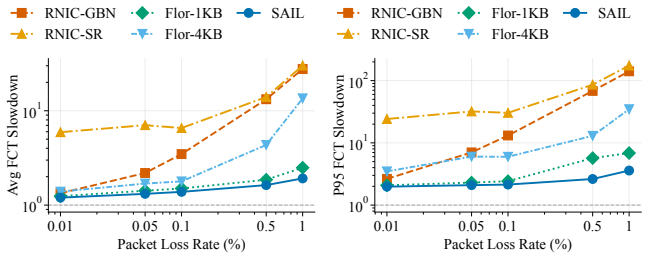
We first evaluate a single sender-receiver pair under saturated demand. Flow sizes follow the Websearch distribution, and the offered load is 1.0. This setting stresses each recovery mechanism near the bottleneck capacity and exposes its achieved goodput and host CPU overhead.

Goodput. Figure 5(a) shows goodput under different random packet loss rates. We see that SAIL consistently achieves the highest goodput, which degrades only slightly as the loss rate increases. This is not surprising because SAIL enables SR with little extra bandwidth consumption and minimal CPU overhead. In particular, SAIL achieves 8.8× higher goodput than GBN and remains 13% higher than Flor-1KB at a 1% loss rate. RNIC-SR achieves the lowest goodput because its bounded window fails to cover the long-haul BDP. GBN performs well at low loss rates, but its goodput degrades quickly as the loss rate increases because each loss triggers a large volume of redundant retransmissions. Flor exhibits performance that varies with chunk sizes. In particular, Flor-1KB generates many chunk operations, and its goodput is limited by the software capacity of issuing and tracking chunk-level work. Flor-4KB performs well at low loss rates, but its goodput degrades once the loss rate reaches 0.1%. This behavior aligns with the transient receive-side RNIC stalls observed in our testbed and is exacerbated by its coarser repair granularity: losing any packet in a chunk necessitates retransmitting the entire chunk.



(a) Average FCT (b) 95th-percentile FCT

Figure 6. Average and 95th-percentile FCT by flow-size range.



(a) Average FCT slowdown (b) 95th-percentile FCT slowdown

Figure 7. Average and 95th-percentile FCT slowdown under different packet loss rates.

CPU overhead. Figure 5(b) reports the sum of sender-side and receiver-side CPU utilization under different random packet loss rates. SAIL keeps CPU utilization far below Flor across all loss rates. Its CPU utilization increases mildly as the loss rate grows, because more losses trigger software repair. This shows the benefit of isolating only the recovery work to host software. At a 1% loss rate, SAIL reduces CPU usage by 92% and 74% compared with Flor-1KB and Flor-4KB, respectively, while achieving higher goodput than both. Flor maintains similarly high CPU utilization across all loss rates because its CPU work lies on the normal data path: host software must chunk messages and issue chunk-level work in proportion to the transmitted data volume. Retransmission accounts for only a small fraction of this software work, so increasing the loss rate has limited impacts on Flor's overall CPU utilization. Flor-4KB consumes less CPU than Flor-1KB because its coarser chunk granularity reduces the frequency of software operations. RNIC-GBN and RNIC-SR consume little CPU because their recovery logic is implemented in RNIC hardware; their host CPU cost mainly comes from posting work requests.

Burst-loss recovery. We further evaluate bursts of B = 8, 16, and 24 packet losses in the same single-pair setting, keeping the overall loss rate at 0.1%. We compare single-slot and two-slot piggybacking with $M = 8$ and $M = 16$. Table 1 shows that increasing M to 16 raises goodput from 17.89 to 22.90 Gbps at $B = 16$. At $B = 24$, two-slot piggybacking with $M = 16$ provides a goodput of 22.34 Gbps, compared with 18.19 Gbps for the default configuration. These results indicate that larger detection windows M and multi-slot piggybacking help reduce burst-induced metadata recovery failures and the resulting GBN fallback overhead.

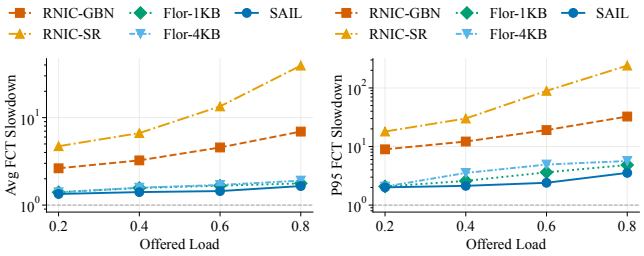
4.3 Flow-Level FCT Improvement

We next evaluate flow-level performance in the multi-pair setting. Each experiment sweeps one parameter and keeps the remaining parameters at their default values, allowing us to test whether SAIL consistently improves flow-level performance across different scenarios.

FCT by flow size. Figure 6 breaks down average and P95 FCT by flow size. We observe that SAIL stays at the best performance for small flows, and its advantage becomes much larger as the flow size increases. For flows no larger than 200 KB, SAIL reduces average FCT by 12–15% and P95 FCT by 3–10% compared with RNIC-GBN. For flows above 5 MB, SAIL reduces average and P95 FCT by 86% and 90% relative to RNIC-GBN, and lowers P95 FCT by 39% relative to Flor-1KB. For small flows, all schemes perform reasonably well because these flows contain only a small number of packets and are less likely to experience packet loss. But large flows are more likely to experience losses, and SAIL avoids letting each loss roll back a long sequence of already delivered packets, achieving much better FCT than GBN. The P95 FCT exhibits a step increase for 50–200 KB flows across most schemes; we attribute this to the higher probability that these larger flows encounter a loss and incur an additional RTT for recovery. RNIC-SR is limited by its bounded receive window, which particularly hurts large flows over the long-haul BDP.

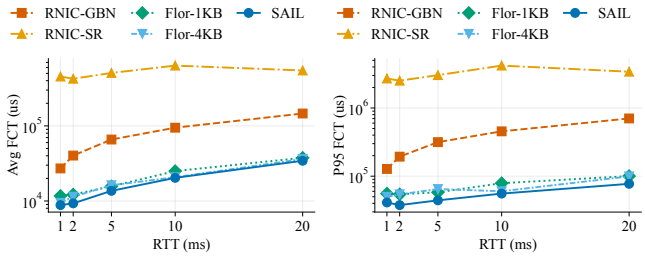
Loss sensitivity. Figure 7 shows the average and P95 FCT slowdown as the packet loss rate increases. SAIL achieves the lowest slowdown across all loss rates, and its slowdown grows much more slowly than the RNIC-only baselines. At a 1% loss rate, SAIL reduces average and P95 FCT slowdown by 93% and 97% relative to RNIC-GBN, respectively, and cuts P95 slowdown by 48% relative to Flor-1KB. RNIC-GBN degrades rapidly as the loss rate increases because higher loss rates trigger more rollback. RNIC-SR avoids rollback, but its receiver-side RNIC only reports the first missing packet in a window, limiting recovery efficiency when multiple losses occur. Flor handles losses more gracefully than RNIC-only recovery, but tail losses still rely on timeout-based retransmission and therefore lead to higher tail latency. In contrast, SAIL's macro-flow loss detection scheme allows subsequent packets from other flows to reveal tail losses without waiting for timeouts.

Load sensitivity. Figure 8 shows FCT slowdown under different offered loads. SAIL still achieves the lowest average and P95 FCT slowdown across all loads, and its slowdown



(a) Average FCT slowdown (b) 95th-percentile FCT slowdown

Figure 8. Average and 95th-percentile FCT slowdown under different offered loads.



(a) Average FCT (b) 95th-percentile FCT

Figure 9. Average and 95th-percentile FCT under different RTTs.

grows much more slowly than the RNIC-only baselines. This result shows that SAIL's recovery path does not amplify congestion under high load. At 0.8 offered load, SAIL reduces average and P95 slowdown by 76% and 89% relative to RNIC-GBN, respectively, and lowers P95 slowdown by 27% relative to Flor-1KB. RNIC-GBN suffers because redundant retransmissions consume bandwidth and increase queueing delay under high load. RNIC-SR avoids redundant retransmission, but its bounded send window becomes a severe bottleneck.

RTT sensitivity. Figure 9 shows the average and P95 FCT as RTT varies. SAIL still achieves the lowest average and tail FCT across all tested RTTs. This result shows that SAIL avoids making loss recovery cost scale with the long-haul BDP. At 20 ms RTT, SAIL reduces average and 95th-percentile FCT by 77% and 89% relative to RNIC-GBN, and by 5% and 22% relative to Flor-4KB, respectively. RNIC-GBN degrades rapidly because each recovery round becomes more expensive as RTT increases. RNIC-SR consistently exhibits high FCT because of its bounded window.

4.4 Resource Usage and Scalability

We evaluate SAIL's data-plane resource usage on a Tofino 1 switch with limited hardware resources. Table 2 reports the hardware resource utilization with the loss-detection window M set to 8. SAIL consumes modest resources across all major resource types. The main stateful resources are the M-slot metadata cache array at S-DCI and the M-slot macro-sequence number tracking array at R-DCI. These resources are independent of the number of RDMA connections and the long-haul BDP. Additionally, the temporary buffering needed to wait for delayed recovery metadata is approximately $M \times$ PMTU. With fixed M and PMTU, the switch buffer requirement also does not grow with link bandwidth or the long-haul BDP.

Higher link rates, such as 100 or 400 Gbps in large-scale datacenter deployments, increase the processing load on switches and host agents. In our Tofino 1 implementation, recirculation may compete with normal forwarding for processing capacity under high load. Under sustained full utilization of the long-haul link, each packet is recirculated once on average, so the additional processing capacity required by recirculation equals the long-haul link rate. In practice, dedicated recirculation ports can be reserved to provide sufficient capacity. Switches with flexible queue management can avoid this recirculation-based queue emulation. At the host, SAIL agents mainly process loss and message-end events, whose frequencies scale linearly with link rate for a fixed loss rate and message-size distribution. CPU demand at higher link rates can thus be estimated by extrapolating the CPU utilization measured in our testbed experiments.

Table 2. SAIL data-plane resource consumption on Tofino 1.

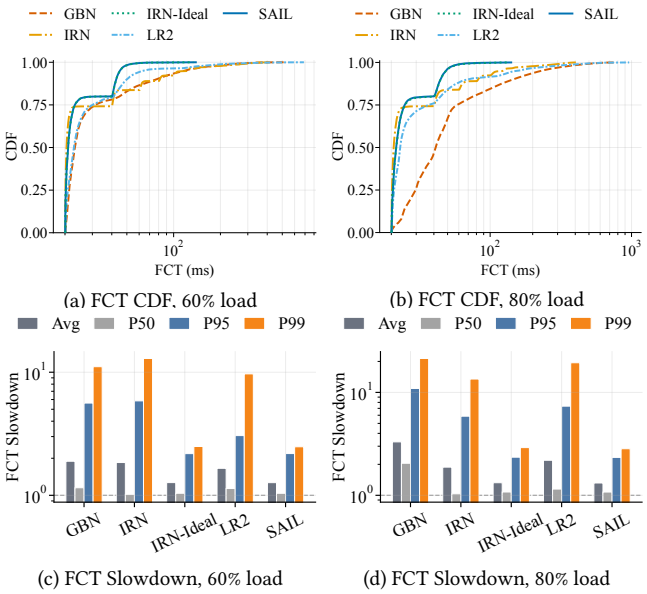
Resource	Gateway	VLIW	ALU	Hash	SRAM
Percentage	9.38%	7.81%	21.62%	12.22%	8.13%

4.5 Large-Scale NS-3 Simulation

Simulation setup. We use NS-3 simulations to evaluate SAIL at a larger scale. The topology contains two datacenters connected by a long-haul link, as shown in Figure 1. Each datacenter uses a $k = 4$ FatTree topology [1] with 16 hosts. Intra-datacenter links run at 100 Gbps, links to the DCI switch run at 400 Gbps, and the long-haul inter-datacenter link runs at 800 Gbps. The long-haul one-way delay is 10 ms, and the long-haul packet loss rate is 0.1%. Intra-datacenter links have 5 μs one-way delay and no random loss. We use DCQCN [58] for congestion control, and enable PFC inside each datacenter to provide lossless intra-datacenter forwarding.

Workloads. The default workload uses Poisson flow arrivals, and flow sizes follow the Websearch distribution. Each cross-datacenter flow uses a randomly selected sender in the left datacenter and a randomly selected receiver in the right datacenter. We evaluate two long-haul offered loads, 60% and 80%. To simulate realistic deployment scenarios, we also inject intra-datacenter background traffic at 50% load.

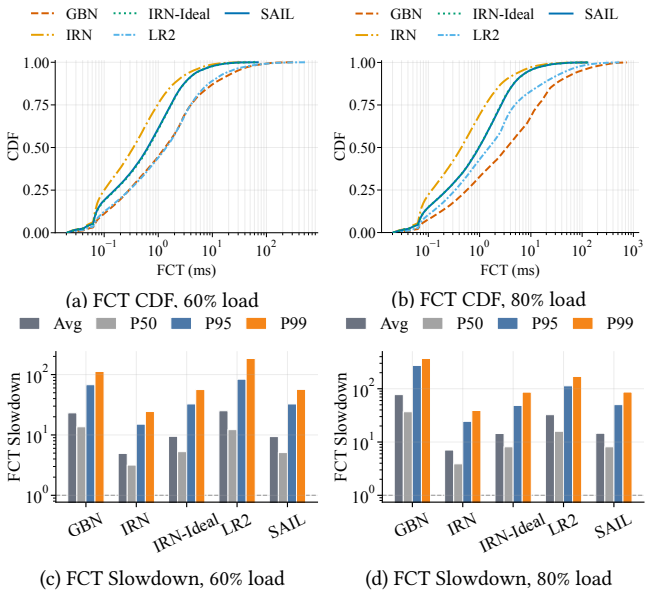
Baselines. We compare SAIL with four baseline schemes: GBN, IRN, IRN-Ideal, and LR2. GBN uses the default Go-Back-N recovery. IRN is an RNIC-based selective-repeat



(a) FCT CDF, 60% load (b) FCT CDF, 80% load

(c) FCT Slowdown, 60% load (d) FCT Slowdown, 80% load

Figure 10. FCT and slowdown of cross-datacenter flows in large-scale NS-3 simulations.



(a) FCT CDF, 60% load (b) FCT CDF, 80% load

(c) FCT Slowdown, 60% load (d) FCT Slowdown, 80% load

Figure 11. FCT and slowdown of sender-side background flows in large-scale NS-3 simulations.

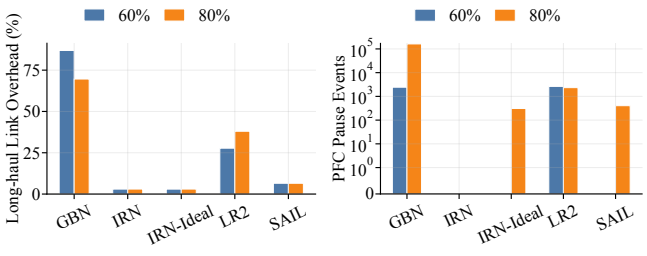
scheme that uses a bounded send window and receiver-side bitmap due to limited RNIC SRAM. We set its window size to 512 packets, matching the maximum SR window supported by current SR-capable commodity RNICs. IRN-Ideal removes IRN's send window and bitmap limits to represent an ideal end-to-end SR scheme. LR2 represents the state-of-the-art DCI-switch-assisted selective repeat design for cross-datacenter RDMA.

Cross-datacenter FCT. Figure 10 reports FCT and slowdown of cross-datacenter flows under 60% and 80% long-haul load. SAIL consistently achieves the lowest average and tail FCT among all schemes, and its CDF nearly overlaps with IRN-Ideal. This is expected because SAIL realizes the same high-level behavior as ideal SR: each loss is reported precisely, and only the missing packet is retransmitted. At 60% load, SAIL reduces p99 FCT by 77.6%, 81.2%, and 74.6% compared with GBN, IRN, and LR2, respectively. At 80% load, the reduction increases to 86.9%, 79.4%, and 85.5%, respectively.

The step observed in SAIL's CDF reflects the extra repair delay paid by flows that experience losses. With a 0.1% packet loss rate, most small flows are loss-free, whereas flows that experience a loss require at least one additional RTT for repair. GBN suffers from long tails because each loss can trigger rollback; this effect becomes worse at 80% load, where redundant retransmissions further congest the long-haul link and shift the whole FCT distribution to the right. LR2 filters redundant retransmissions at the sender-side DCI switch, allowing it to approximate SR on the long-haul link and avoid the severe load sensitivity of GBN. However, LR2 still relies on RNIC-triggered GBN recovery and receiver-side switch buffering, so its overall performance remains worse than ideal SR schemes. IRN avoids redundant retransmission, but its bounded window cannot cover the long-haul BDP, causing bursty window-limited transmission and step-like growth in the CDF. Interestingly, IRN achieves the best p50 FCT under both loads because most Websearch flows are small and are not constrained by the send window. The window limit throttles large flows and reduces network utilization, which benefits small flows but hurts tail FCT.

Impact on intra-datacenter traffic. Figure 11 shows the FCT and slowdown of sender-side intra-datacenter background flows under 60% and 80% long-haul load. SAIL keeps the overall performance of background flows close to IRN-Ideal, and is only worse than IRN. This is because SAIL's repair traffic is proportional to missing payloads and does not inject redundant traffic into the sender-side datacenter. At 60% load, SAIL reduces background-flow p99 FCT by 82.7% and 82.1% compared with GBN and LR2, respectively. At 80% load, the p99 reduction is 91.4% over GBN and 81.8% over LR2. GBN hurts local traffic because redundant retransmissions traverse the sender-side datacenter. LR2 drops redundant retransmissions at the sender-side DCI switch, but these packets can still consume part of the local path before being filtered. IRN has little impact on background traffic because its bounded window limits the amount of in-flight data, reducing local congestion at the cost of low cross-datacenter flow performance.

Recovery behavior breakdown. To further explain the FCT improvements above, Figure 12 reports two metrics related to recovery behavior and network conditions under 60% and 80% long-haul load. Figure 12(a) shows the long-haul link overhead of each scheme, defined as the amplification from



(a) Long-haul link overhead (b) PFC pause events

Figure 12. Recovery overhead and congestion indicators in large-scale NS-3 simulations.

cross-datacenter application payload bytes to the total bytes carried over the long-haul link. SAIL reduces long-haul overhead by 92.5%/90.6% compared with GBN and by 76.5%/82.8% compared with LR2 at 60%/80% load, demonstrating the benefit of precise retransmission. Its overhead is only slightly higher than IRN-Ideal due to the per-packet MSH inserted by SAIL, which does not materially affect performance on the non-saturated long-haul link. Figure 12(b) reports the PFC pause events observed at the sender-side data center. Similar to IRN-Ideal, SAIL triggers no PFC pauses at 60% load and only a small number of pauses at 80% load. In contrast, GBN and LR2 already trigger many pause events at 60% load because redundant retransmissions increase local congestion inside the datacenter. IRN triggers no PFC pauses because its bounded window naturally limits the amount of in-flight packets.

Extra switch buffer and data plane state usage. We separately measure the peak extra DCI switch buffer demand and data-plane state usage of LR2 and SAIL. Here, the extra switch buffer refers to the logical packet buffering required by the recovery mechanism beyond normal switch forwarding queues. LR2 requires 2469 MB and 2875 MB of extra switch buffer at 60% and 80% load, respectively, whereas SAIL uses only 31.2 KB under both loads. This large gap comes from their different buffering roles. LR2 uses the receiver-side DCI switch as a reorder buffer for out-of-order post-loss packets, so its buffer demand can grow with the long-haul BDP and the amount of in-flight traffic. In contrast, SAIL only uses a small switch queue for ordered release, whose size is independent of the BDP and is determined by the loss-detection window M.

For data-plane state, LR2 consumes 3.5 MB and 4.6 MB of SRAM at 60% and 80% load, respectively, whereas SAIL uses only 360 B under both loads. This is because LR2 maintains per-flow recovery state, so its SRAM usage grows with the number of active flows. SAIL, instead, tracks a single macro-flow and caches only recent packet metadata for loss recovery, whose size is also determined by M. Together, these results show that SAIL avoids both BDP-scale extra buffering and flow-proportional switch state.

5 Discussion

Supported RDMA operations. Our current design focuses on RDMA WRITE and SEND/RECV, which are widely used by RDMA-based storage, RPC, and collective communication systems [7, 19, 20, 39, 47]. These operations share similar one-way data-transfer logic, whereas RDMA READ and atomic operations require protection of both request and response traffic. To support atomic operations, the piggybacked metadata could be extended with the operation-specific fields needed for DCI switches to reconstruct lost requests and responses directly. READ requests could be handled similarly, while READ responses carry bulk data in the reverse direction and could be protected by deploying SAIL's recovery logic symmetrically in that direction. Supporting these operations would also require additional logic to preserve their ordering and completion semantics. We leave full support for these operations to future work.

Interaction with congestion control. SAIL is designed to reduce the cost of packet loss, not to eliminate the root causes of loss. It is most effective for non-persistent losses, including random losses and moderate loss bursts. If the long-haul path experiences persistent congestion, repair traffic may become substantial, and the backup UC QP may itself become a bottleneck. Therefore, SAIL should be used together with congestion control and traffic engineering mechanisms that reduce the probability of loss. In practice, SAIL complements such mechanisms: congestion control and traffic engineering reduce the loss rate, while SAIL reduces the performance penalty when losses still occur.

Losses on the intra-datacenter segments. SAIL targets losses on the long-haul segment of cross-datacenter RDMA flows because this segment is more exposed to random losses caused by long-distance optical impairments, and because deploying PFC over long-haul links is impractical due to the large headroom required by the large BDP. In contrast, intra-datacenter segments are short and typically protected by PFC. However, packet losses may still occur in rare cases due to packet corruption or link failures, affecting SAIL's behavior in two ways. First, losing an RDMA data packet prevents SAIL from masking the PSN gap from the RNIC, triggering the RNIC's native GBN recovery without compromising correctness. Second, losing a notification packet generated by the *R-DCI* may affect the *R-agent*'s data repair and completion validation. Such notification losses can be addressed with a lightweight reliable notification mechanism between the *R-DCI* and the host. In brief, each notification packet carries a sequence number, allowing the host to detect missing notifications and send ACK/NACK signals. The *R-DCI* temporarily retains each notification packet (e.g., through recirculation in Tofino 1), retransmits it upon receiving a NACK or a timeout, and releases it upon receiving an ACK, in a manner similar to LR2 [29].

Packet reordering on the inter-datacenter segment. SAIL's macro-flow loss detection assumes that packets within the same macro-flow traverse the same long-haul path and arrive at the *R-DCI* in order. This avoids confusing path-induced reordering with packet loss. When multiple long-haul paths are available, SAIL can maintain an independent macro-flow for each path. Specifically, the sender-side DCI switch first assigns each RDMA flow to a long-haul path according to the routing or load-balancing policy, and then aggregates flows mapped to the same path into the corresponding macro-flow. The receiver-side DCI switch performs loss detection independently for each path. This makes the switch state grow with the number of long-haul paths. This scaling remains practical for inter-datacenter communication, where production systems typically use a limited set of traffic-engineering paths [10, 22]. In rare cases, packet reordering may also occur within a single long-haul path. However, loss detection at the *R-DCI* is delayed by M packets via SAIL's delayed-piggyback, so reordering within this window does not trigger false loss detection. Increasing M can improve tolerance to intra-path reordering.

Partial host deployment. SAIL can be incrementally deployed on a subset of hosts in a datacenter. For connections whose two endpoints do not both run SAIL agents, the DCI switches should not mask losses with dummy packets, because the receiver would not have the software support needed to repair dummy-filled memory regions. To this end, the R-DCI maintains two tables that record SAIL-enabled host IPs in the two datacenters, respectively. It performs dummy packet insertion only when both the source and destination IPs match the corresponding tables. Other traffic is forwarded normally and continues to rely on the RNIC's native recovery mechanism. Meanwhile, the SAIL agent logic is enabled only when both endpoints run the agent.

6 Related Work

RDMA loss recovery. Besides the loss-recovery mechanisms discussed in §1.2, recent work also improves RDMA reliability by changing the transport protocol or the reliability mechanism. DCP [25] builds a lossless control plane in the network to detect losses and notify RNICs, allowing RNICs to support selective repeat without maintaining bitmap-based packet tracking. However, DCP requires network-wide switch support and coordinated RNIC changes, and mainly targets congestion-induced losses. SRD [43], UEC/UET [48], and Falcon [44] design new hardware transport protocols to provide more efficient reliability. These designs require new standards, NIC capabilities, or deployment ecosystems. SAIL instead improves loss recovery for existing RDMA deployments with commodity RNICs.

Long-haul RDMA. The growing demand for communication across datacenters has motivated recent studies on extending RDMA to long-haul datacenter interconnects. Swing [4] and Bifrost [12] improve flow control with PFC-Relay and downstream-driven in-flight packet estimation, respectively, reducing the switch-buffer demand of long-haul lossless network. BiCC [49], Themis [34], and LSCC [28] improve long-haul RDMA congestion control, reducing the unfairness and inefficient rate control caused by delayed congestion signals. LCMP [54] addresses long-haul RDMA load balancing by adapting traffic distribution across heterogeneous inter-datacenter paths.

Coding-based reliability. Coding-based approaches provide another way to handle packet loss beyond retransmission-based reliability. They add redundant information to transmitted traffic so that receivers can recover from some packet losses without waiting for a long-RTT retransmission feedback loop. Recent systems have explored such mechanisms for RDMA communication over lossy long-haul networks [21, 59]. However, coding-based recovery consumes extra bandwidth and computation even when packets are delivered successfully, and its loss tolerance is bounded by the amount of redundancy and the coding scheme. For RDMA, it may also require an additional coding layer or programmable endpoint support on hardware.

In-network support for RDMA. Programmable switches have enabled a broad range of in-network support for RDMA networks by exposing network-side signals and executing lightweight data-plane logic. Prior systems use such capabilities to improve RDMA congestion control [3, 26], load balancing [5, 27, 45], security defense [53, 56], and network monitoring [23, 50]. These systems exploit information available inside the network to address performance, robustness, security, or observability problems that are difficult to solve at end hosts alone. SAIL is inspired by these systems. It uses programmable DCI switches to offload loss detection.

7 Conclusion

We present SAIL, a scalable in-network-assisted loss recovery mechanism for cross-datacenter RDMA. SAIL coordinates DCI switches and end-host software through macro-flow loss detection, metadata piggybacking, dummy packet injection, and precise software repair, enabling selective repeat without modifying commodity RNICs. Testbed experiments and large-scale simulations demonstrate higher goodput and lower FCT with low CPU overhead and modest switch resource usage.

Acknowledgments

We thank our anonymous shepherd and reviewers for their valuable comments. This work was supported by the National Natural Science Foundation of China (No. 62221003). Yuan Yang and Mingwei Xu are the corresponding authors.

References

[1] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A Scalable, Commodity Data Center Network Architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*. ACM, 63–74.

[2] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data Center TCP (DCTCP). In *Proceedings of the ACM SIGCOMM 2010 Conference*. ACM, 63–74.

[3] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minlan Yu, and Michael Mitzenmacher. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. ACM, 662–680.

[4] Yanqing Chen, Chen Tian, Jiaqing Dong, Song Feng, Xu Zhang, Chang Liu, Peiwen Yu, Nai Xia, Wanchun Dou, and Guihai Chen. 2023. Swing: Providing Long-Range Lossless RDMA via PFC-Relay. *IEEE Transactions on Parallel and Distributed Systems* 34, 1 (2023), 63–75.

[5] Haotian Deng, Yuan Yang, Menghao Zhang, and Mingwei Xu. 2024. CAVER: Enhancing RDMA Load Balancing by Hunting Less-Congested Paths. In *Proceedings of the ACM SIGCOMM 2024 Conference: Posters and Demos*. ACM, 39–41.

[6] DPDK Project. 2026. Data Plane Development Kit Programmer's Guide. https://doc.dpdk.org/guides/prog_guide/. (2026). Accessed: 2026-06-11.

[7] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed AI Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, 57–70.

[8] Google Cloud. 2024. Using Dual-Region Buckets in Google Cloud Storage. https://cloud.google.com/blog/products/storage-data-transfer/using-dual-region-buckets-in-google-cloud-storage. (2024).

[9] Thomas R. Henderson, Mathieu Lacage, George F. Riley, Craig Dowell, and Joseph Kopena. 2008. Network Simulations with the ns-3 Simulator. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*. ACM, 527.

[10] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. 2013. Achieving High Utilization with Software-Driven WAN. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*. ACM, 15–26.

[11] Chi-Yao Hong, Subhasree Mandal, Mohammad Al-Fares, Min Zhu, Richard Alimi, Kondapa Naidu B., Chandan Bhagat, Sourabh Jain, Jay Kaimal, Shiyu Liang, Kirill Mendelev, Steve Padgett, Faro Rabe, Saikat Ray, Malveeka Tewari, Matt Tierney, Monika Zahn, Jonathan Zolla, Joon Ong, and Amin Vahdat. 2018. B4 and After: Managing Hierarchy, Partitioning, and Asymmetry for Availability and Scale in Google's Software-Defined WAN. In *Proceedings of the ACM SIGCOMM 2018 Conference*. ACM, 74–87.

[12] Chengyuan Huang, Feiyang Xue, Peiwen Yu, Xiaoliang Wang, Yanqing Chen, Tao Wu, Lei Han, Zifa Han, Bingquan Wang, Xiangyu Gong, Chen Tian, Wanchun Dou, Guihai Chen, and Hao Yin. 2024. Minimizing Buffer Utilization for Lossless Inter-DC Links. *IEEE/ACM Transactions on Networking* 32, 6 (2024), 4960–4975.

[13] Peihao Huang, Guo Chen, Xin Zhang, Can Liu, Hongyu Wang, Huijun Shen, Ying Bian, Yuanwei Lu, Zhenyuan Ruan, Bojie Li, et al. 2025. Fast and Scalable Selective Retransmission for RDMA. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.

[14] IEEE. 2011. IEEE Std 802.1Qbb-2011: IEEE Standard for Local and Metropolitan Area Networks—Media Access Control (MAC) Bridges and Virtual Bridged Local Area Networks—Amendment 17: Priority-based Flow Control. https://standards.ieee.org/ieee/802.1Qbb/4361/. (2011).

[15] Intel. 2016. Intel Tofino 3.2 Tbps, 2 Pipelines. https://www.intel.com/content/www/us/en/products/sku/218641/intel-tofino-3-2-tbps-2-pipelines/specifications.html. (2016).

[16] Intel. 2018. Intel Tofino 2 6.4 Tbps, 4 Pipelines. https://www.intel.com/content/www/us/en/products/sku/218647/intel-tofino-2-6-4-tbps-4-pipelines/specifications.html. (2018).

[17] Intel. 2021. P4_16 Intel Tofino Native Architecture – Public Version. (2021). https://github.com/barefootnetworks/Open-Tofino/blob/master/PUBLIC_Tofino-Native-Arch.pdf Accessed: 2026-06-11.

[18] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, Jonathan Zolla, Urs H"olzle, Stephen Stuart, and Amin Vahdat. 2013. B4: Experience with a Globally-Deployed Software Defined WAN. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*. ACM, 3–14.

[19] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA Efficiently for Key-Value Services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*. ACM, 295–306.

[20] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. FaSST: Fast, Scalable and Simple Distributed Transactions with Two-Sided (RDMA) Datagram RPCs. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, 185–201.

[21] Mikhail Khalilov, Siyuan Shen, Marcin Chrapek, Tiancheng Chen, Kenji Nakano, Nicola Mazzoletti, Peter-Jan Gootzen, Salvatore Di Girolamo, Rami Nudelman, Gil Bloch, Jithin Jose, Abdul Kabbani, Sreevatsa Anantharamu, Jie Zhang, Konstantin Taranov, Zhuolong Yu, Scott Moe, Mahmoud Elhaddad, and Torsten Hoefler. 2025. SDR-RDMA: Software-Defined Reliability Architecture for Planetary Scale RDMA Communication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. ACM, 1223–1239.

[22] Umesh Krishnaswamy, Rachee Singh, Paul Mattes, Paul-Andre C. Bissonnette, Nikolaj S. Bjørner, Zahira Nasrin, Sonal Kothari, Prabhakar Reddy, John Abeln, Srikanth Kandula, Himanshu Raj, Luis Ir'un-Briz, Jamie Gaudette, and Erica Lan. 2023. OneWAN is Better Than Two: Unifying a Split WAN Architecture. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 515–529.

[23] Guanyu Li, Menghao Zhang, Cheng Guo, Han Bao, Mingwei Xu, Hongxin Hu, and Fenghua Li. 2022. IMap: Fast and Scalable In-Network Scanning with Programmable Switches. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 667–681.

[24] Qiang Li, Yixiao Gao, Xiaoliang Wang, Haonan Qiu, Yanfang Le, Derui Liu, Qiao Xiang, Fei Feng, Peng Zhang, Bo Li, Jianbo Dong, Lingbo Tang, Hongqiang Harry Liu, Shaozong Liu, Weijie Li, Rui Miao, Yaohui Wu, Zhiwu Wu, Chao Han, Lei Yan, Zheng Cao, Zhongjie Wu, Chen Tian, Guihai Chen, Dennis Cai, Jinbo Wu, Jiaji Zhu, Jiesheng Wu, and Jiwu Shu. 2023. Flor: An Open High Performance RDMA Framework over Heterogeneous RNICs. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, 931–948.

[25] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. 2025. Revisiting RDMA Reliability for Lossy Fabrics. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. ACM, 85–98.

[26] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: High Precision Congestion Control. In *Proceedings of the ACM SIGCOMM 2019 Conference*. ACM, 44–58.

[27] Xiangzhou Liu, Wenxue Li, Zihao Wang, and Kai Chen. 2026. Enabling Packet Spraying over Commodity RNICs with In-Network Support. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 693–709.

[28] Minfei Long, Jiangping Han, Wentao Wang, Jiayu Yang, and Kaiping Xue. 2024. LSCC: Link-segmented congestion control for RDMA in cross-datacenter networks. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.

[29] Minfei Long, Jiangping Han, Kaiping Xue, Jinhao Liu, and Jian Li. 2026. LR2: Accelerating Long-Distance RDMA Recovery via In-Network Retransmission Decoupling. In *IEEE INFOCOM 2026-IEEE Conference on Computer Communications*. IEEE, 1–10.

[30] Yuanwei Lu, Guo Chen, Zhenyuan Ruan, Wencong Xiao, Bojie Li, Jiansong Zhang, Yongqiang Xiong, Peng Cheng, and Enhong Chen. 2017. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proceedings of the First Asia-Pacific Workshop on Networking*. 22–28.

[31] Kai Lv, Jinyang Li, Pengyi Zhang, Heng Pan, Luyang Li, Shuihai Hu, Zhenyu Li, Gaogang Xie, Jingbin Zhou, and Kun Tan. 2025. Omnidma: Scalable rdma transport over wan. In *Proceedings of the 9th Asia-Pacific Workshop on Networking*. 135–141.

[32] Microsoft. 2026. Azure Region Pairs and Nonpaired Regions. https://learn.microsoft.com/en-us/azure/reliability/regions-paired. (2026). Accessed: 2026-06-11.

[33] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. 2018. Revisiting Network Support for RDMA. In *Proceedings of the 2018 ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 313–326.

[34] Zihan Niu, Menghao Zhang, Jue Zhang, Renjie Xie, Yuan Yang, and Xiaohe Hu. 2025. THEMIS: Addressing Congestion-Induced Unfairness in Long-Haul RDMA Networks. In *Proceedings of the 2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 1–13.

[35] NVIDIA. 2021. Mellanox ConnectX-7 Product Brief. (2021). https://www.nvidia.com/content/dam/en-zz/Solutions/networking/ethernet-adapters/connectx-7-datasheet-Final.pdf

[36] NVIDIA. 2023. RDMA Aware Networks Programming User Manual. https://docs.nvidia.com/networking/display/RDMAAwareProgrammingv17. (2023).

[37] NVIDIA. 2026. BlueField-3 Networking Platform User Guide: Supported Interfaces. (2026). https://docs.nvidia.com/networking/display/BF3DPU/Supported-Interfaces Accessed: June 2026.

[38] NVIDIA. 2026. MTU Considerations for RoCE Based Applications. https://enterprise-support.nvidia.com/s/article/mtu-considerations-for-roce-based-applications. (2026). Accessed: 2026-06-11.

[39] NVIDIA. 2026. NVIDIA Collective Communications Library (NCCL). https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/. (2026). Accessed: 2026-06-11.

[40] NVIDIA. 2026. NVIDIA ConnectX-6 Dx Firmware Release Notes: Changes and New Feature History. (2026). https://docs.nvidia.com/networking/display/ConnectX6DxFirmwarev22344000/Changes+and+New+Feature+History Accessed: June 2026.

[41] NVIDIA. 2026. NVIDIA Spectrum-X Ethernet Networking Platform. https://www.nvidia.com/en-us/networking/spectrumx/. (2026). Accessed: 2026-06-11.

[42] SAIL. 2026. SAIL repository. https://github.com/lixiao19/sail. (2026).

[43] Leah Shalev, Hani Ayoub, Nafea Bshara, and Erez Sabbag. 2020. A Cloud-Optimized Transport Protocol for Elastic and Scalable HPC. *IEEE Micro Special Issue on Commercial Products* (2020). https://www.amazon.science/publications/a-cloud-optimized-transport-protocol-for-elastic-and-scalable-hpc

[44] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. 2025. Falcon: A Reliable, Low Latency Hardware Transport. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. ACM, 249–264.

[45] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. 2023. Network Load Balancing with In-Network Reordering Support for RDMA. In *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 816–831.

[46] Haoze Song, Yongqi Wang, Xusheng Chen, Hao Feng, Yazhi Feng, Xieyun Fang, Heming Cui, and Linghe Kong. 2025. K2: On Optimizing Distributed Transactions in a Multi-region Data Store with TrueTime Clocks. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1756–1769.

[47] Yacine Taleb, Ryan Stutsman, Gabriel Antoniu, and Toni Cortes. 2018. Tailwind: Fast and Atomic RDMA-based Replication. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, 851–863.

[48] Ultra Ethernet Consortium. 2025. Ultra Ethernet Specification 1.0. (2025). https://ultraethernet.org/uec-1-0-spec

[49] Zirui Wan, Jiao Zhang, Mingxuan Yu, Junwei Liu, Jun Yao, Xinghua Zhao, and Tao Huang. 2024. BiCC: Bilateral Congestion Control in Cross-datacenter RDMA Networks. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. IEEE, 1381–1390.

[50] Shicheng Wang, Menghao Zhang, Xiao Li, Qiyang Peng, Haoyuan Yu, Zhiliang Wang, Mingwei Xu, Xiaohe Hu, Jiahai Yang, and Xingang Shi. 2025. Hawkeye: Diagnosing RDMA Network Performance Anomalies with PFC Provenance. In *Proceedings of the ACM SIGCOMM 2025 Conference*. ACM, 481–495.

[51] Wentao Wang, Jiangping Han, Kaiping Xue, Jian Li, Kunpeng Ding, and Ruidong Li. 2025. NetRT: Enhancing RDMA with Retransmission Offloading in Data Center Networks. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.

[52] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchen Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. 2023. SRNIC: A Scalable Architecture for RDMA NICs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 1–14.

[53] Jiarong Xing, Kuo-Feng Hsu, Yiming Qiu, Ziyang Yang, Hongyi Liu, and Ang Chen. 2022. Bedrock: Programmable Network Support for Secure RDMA Systems. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2585–2600.

[54] Dong-Yang Yu, Yuchao Zhang, Xiaodi Wang, Jun Wang, Wenfei Wu, Haipeng Yao, Wendong Wang, and Ke Xu. 2026. LCMP: Distributed Long-Haul Cost-Aware Multi-Path Routing for Inter-Datacenter RDMA Networks. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. ACM.

[55] Gaoxiong Zeng, Wei Bai, Chuanxiong Guo, Zhi-Li Zhang, and Suresh Singh. 2022. Congestion Control for Cross-Datacenter Networks. *IEEE/ACM Transactions on Networking* 30, 5 (2022), 2073–2087.

[56] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qi Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating Volumetric DDoS Attacks with Programmable Switches. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. The Internet Society.

[57] Yang Zhou, Zhongjie Chen, Ziming Mao, ChonLam Lao, Shuo Yang, Pravein Govindan Kannan, Jiaqi Gao, Yilong Zhao, Yongji Wu, Kaichao You, Fengyuan Ren, Zhiying Xu, Costin Raiciu, and Ion Stoica. 2025. An Extensible Software Transport Layer for GPU Networking. *arXiv preprint arXiv:2504.17307* (2025). https://arxiv.org/abs/2504.17307

[58] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, 523–536.

[59] Tianyu Zuo, Tao Sun, Shuyong Zhu, Wenxiao Li, Lu Lu, Zongpeng Du, and Yujun Zhang. 2024. LoWAR: Enhancing RDMA over Lossy WANs with Transparent Error Correction. In *Proceedings of the 2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS '24)*. IEEE, 1–10.